

NGS2026 — Pipeline Edge AI v2

New Generation Sensors · Smart City Vehicle & License Plate Recognition
Maggio 2026

Sommario esecutivo

Questo documento descrive la **seconda versione** della pipeline operativa NGS2026 per il rilevamento, tracking, classificazione veicoli e lettura targhe. Rispetto alla v1, sono stati sostituiti i due componenti più deboli (plate detection e OCR) con tecnologie significativamente più performanti, validate nei benchmark documentati nei report tecnici precedenti.

La pipeline gira interamente on-device su **Raspberry Pi 5 + Hailo-8 AI HAT+**, senza dipendenze di rete, ed elabora video offline producendo un file JSON con tutte le rilevazioni e un video annotato.

Modifiche chiave rispetto a v1:

Componente	v1	v2
Plate detection	tiny_yolov4_license_plates.hef (F1=0.638)	license_plate_finetune_v1n.hef (F1=0.955)
OCR	EasyOCR (16.7% exact match su targhe miste UE)	fast-plate-ocr cct-xs-v1 (86.0%)
Logica anti-duplicati	nessuna	majority voting + merge by plate con timeout temporale

1. Architettura della pipeline

La pipeline è strutturata in **5 stadi sequenziali**:

Stage 1 — Vehicle detection (Hailo-8)

- Modello:** yolov11s.hef (small, ~10MB) compilato per Hailo-8

- **Input:** frame intero 640x640 RGB
- **Output:** bbox + classe + confidence per ogni veicolo rilevato
- **Classi (6):** bus, car, pickup, truck, van, motorcycle
- **Confidence threshold:** 0.3

Stage 2 — Plate detection (Hailo-8)

- **Modello:** `license_plate_finetune_v1n.hef` (nano YOLOv11, ~2MB) compilato per Hailo-8
- **Input:** crop del singolo veicolo (non frame intero), letterboxed a 640x640 RGB
- **Output:** bbox della targa in coordinate del crop, poi riportata in coordinate assolute del frame
- **Scelta detection:** massima confidence per veicolo
- **Confidence threshold:** 0.3

Razionale “plate detection sui crop”: applicare v1n al crop del veicolo (non al frame intero) riduce drasticamente i falsi positivi su cartelli, scritte pubblicitarie, numeri civici, ecc.

Stage 3 — Tracking (CPU)

- **Algoritmo:** ByteTrack via `supervision` library
- **Funzione:** assegna un `tracker_id` stabile a ciascun veicolo nel tempo
- **Limite noto:** in caso di occlusioni prolungate (auto che passa dietro un'altra), il tracker può perdere l'ID e crearne uno nuovo quando il veicolo riappare. Mitigato dallo Stage 6 (merge by plate).

Stage 4 — OCR (CPU, ONNX Runtime)

- **Modello:** `fast-plate-ocr` con `cct-xs-v1-global-model` (~2MB ONNX)
- **Input:** crop della targa in formato RGB (3 canali)
- **Tempo:** ~21 ms per crop su CPU RPi5
- **Output:** stringa di caratteri letta (A-Z, 0-9)

Nota tecnica: il modello richiede input RGB. Il codice converte esplicitamente `cv2.cvtColor(crop, cv2.COLOR_BGR2RGB)` prima dell'inferenza.

Stage 5 — Majority voting per tracker_id

A fine elaborazione del video (offline batch), per ogni `tracker_id` rilevato:

- **Classe finale** = la più votata tra tutte le rilevazioni di quel `tracker_id`

- **Targa finale** = la più votata tra tutte le letture OCR di quel `tracker_id` (esclusi i frame con OCR vuoto)
- Se nessuna lettura OCR esiste per quel `tracker_id`, la targa è marcata come `"unknown"`

Questa logica sfrutta la ridondanza delle letture: anche con OCR al ~86% su singolo frame, ripetere la lettura su 50-100 frame per veicolo porta l'accuratezza reale molto più in alto, perché gli errori non si ripetono identici frame-by-frame.

Stage 6 — Merge by plate with timeout temporale

Logica aggiuntiva per gestire il problema delle **occlusioni** (tracker che perde l'ID quando un veicolo passa dietro un altro e poi viene re-istanziato come ID nuovo quando riappare).

Algoritmo:

1. Raggruppa i `tracker_id` per targa votata identica (esclusi gli `unknown`)
2. Per ogni gruppo, ordina i `tracker_id` per primo frame di apparizione
3. Greedy clustering temporale: due `tracker_id` consecutivi vengono uniti se il gap tra l'ultima apparizione del precedente e la prima del successivo è $< \text{MERGE_MAX_GAP_SECONDS}$ (default: 15 secondi)
4. Se il gap è maggiore, i due sono trattati come **transiti distinti** dello stesso veicolo

Razionale:

- Un veicolo occluso per occlusioni brevi (1-3 secondi) viene unificato conteggio corretto
- Un veicolo che esce dalla scena e rientra molto dopo (es. uscendo e rientrando da un parcheggio) viene contato come **due eventi distinti** riflette correttamente i transiti

2. Modalità di esecuzione

Il codice supporta due modalità configurabili (`EXECUTION_MODE`):

`sequential` (default)

1. **Passata 1:** yolo11s su TUTTI i frame del video
2. **Passata 2:** v1n sui crop veicolo di tutti i frame
3. Le passate sono separate: prima carico il modello yolo11s, lo eseguo, lo deconfiguro, poi carico v1n

Vantaggi: più veloce (le riconfigurazioni Hailo costano), ottimo per batch offline.

alternating

Per ogni frame:

1. Configura yolo11s, inferenza, deconfigura
2. Configura v1n, inferenza, deconfigura

Vantaggi: simula meglio una pipeline real-time. Più lento per via delle riconfigurazioni continue.

3. Output prodotti

La pipeline produce due artefatti per ogni esecuzione:

output.mp4

Video annotato a fps originale, con:

- **Bbox veicolo:** rettangolo colorato per classe (bus/car/pickup/truck/van/motorcycle)
- **Label sopra il veicolo:** ID{tracker_id} {classe} {confidence}
- **Bbox targa:** rettangolo arancione
- **Sopra la targa:** lettura finale votata [CM128ZK] in giallo
- **Sotto la targa:** lettura OCR del frame corrente (può cambiare durante il video)

detections.json

Struttura:

```
{
  "video": "/percorso/video.mp4",
  "fps": 26.3,
  "frames_total": 715,
  "execution_mode": "sequential",
  "timings_seconds": {
    "detection_total": 32.5,
    "tracking": 0.6,
    "ocr": 5.8
  },
}
```

```

"vehicle_count": 31,
"vehicles": [
  {
    "tracker_id": 4,
    "class": "car",
    "plate": "WG2119",
    "n_frames_seen": 55,
    "n_plate_reads": 49
  },
  ...
],
"frames": [
  { "frame": 0, "vehicles": [...] },
  ...
]
}

```

- **Riepilogo per veicolo:** classe, targa votata, numero frame in cui appare, numero letture OCR valide
- **Dettaglio per frame:** ogni veicolo con bbox, classe corrente, targa OCR del frame
- **Stampa testuale a fine esecuzione:** tabella riepilogativa con ID, classe, targa, frame, OCR_OK

4. Configurazione

Tutti i parametri sono in cima allo script. I principali:

Parametro	Default	Significato
VIDEO_PATH	-	Path al video di input
SMALL_HEF	/home/pi/yolov11s.hef	Path al modello vehicle detection
PLATE_HEF	license_plate_finetune_v1n.hef	Path al modello plate detection
OCR_MODEL_NAME	cct-xs-v1-global-model	Nome modello fast-plate-ocr

EXECUTION_MODE	sequential	sequential o alternating
VEHICLE_CONF	0.3	Soglia confidence vehicle detection
PLATE_CONF	0.3	Soglia confidence plate detection
MERGE_MAX_GAP_SECONDS	15.0	Gap temporale massimo per merge by plate

5. Test su video reale

Setup del test

- **Video:** scena stradale urbana con traffico denso (715 frame @ 26.3 fps, ~27 secondi)
- **Hardware:** Raspberry Pi 5 + Hailo-8 AI HAT+
- **Modalità:** sequential
- **Merge timeout:** 15 secondi

Risultati di performance

Metrica	Valore
Frame totali processati	715
Tempo detection (vehicle + plate, sequential)	31.6 s
Tempo tracking (ByteTrack)	0.6 s
Tempo OCR (fast-plate-ocr)	5.8 s
Tempo totale	38.0 s
Rapporto realtime	~0.7x (su 27s di video, ne servono 38)

Risultati di conteggio

Metrica	Valore
Veicoli reali nella scena (stima visiva)	20-25
tracker_id pre-merge	34
Veicoli finali post-merge	31
ID assorbiti dal merge by plate	3

Interpretazione:

- La pipeline sovrastima leggermente il conteggio (~24% in più rispetto alla stima visiva)
- Il merge by plate ha unificato 3 ID (es. quattro istanze separate di WG2119 un unico veicolo)
- Le sovrastime residue sono dovute principalmente a:
 - **Letture OCR instabili** che producono targhe “vicine ma non identiche” (es. 3336 , 333300 , 33360) il merge by plate non le unifica perché richiede match esatto
 - **Tracker_id con OCR unknown** (3 nel test) impossibile mergiare senza targa
 - **Veicoli con poche letture** (1-2 frame) tipicamente passaggi rapidi o false detection

Note sul contesto del test

Il video di test è uno **scenario stress** per la pipeline:

- Camera bassa (~1-1.5m altezza) forte sovrapposizione tra veicoli
- Traffico denso occlusioni frequenti
- Condizioni di illuminazione mista (notturne con fari)
- Targhe non europee font diverso dal training dataset di fast-plate-ocr

Nel caso d'uso reale NGS (camera a 2-3m altezza, semaforica, traffico italiano), molti di questi fattori sono attenuati, quindi ci si aspettano risultati significativamente migliori.

6. Limiti noti e prospettive

Limiti identificati nei test

1. **Occlusioni prolungate:** il tracker perde ID quando un veicolo è occluso per più frame. Mitigato dal merge by plate, ma solo se l'OCR legge la stessa targa in entrambi i tronconi.
2. **Letture OCR vicine ma non identiche:** il merge by plate richiede match esatto. Letture come 3336 vs 33360 non vengono unificate anche se sono probabilmente lo stesso veicolo. Soluzione futura: **fuzzy matching** con distanza di Levenshtein ≤ 2 .
3. **Veicoli con OCR unknown** : tracker_id che non hanno mai prodotto una lettura OCR (3 nel test) non possono essere unificati. È un caso strutturalmente irrisolvibile senza altri segnali (es. tracking visivo più robusto).
4. **Tempo real-time non raggiunto in modalità sequential:** ~38s per 27s di video. Ottimizzazioni possibili: parallelizzazione I/O frame, batching delle inferenze OCR.

Miglioramenti possibili (non implementati)

Miglioramento	Difficoltà	Beneficio atteso
Tuning ByteTrack (lost_track_buffer più alto)	Bassa	Riduce ID spuri da occlusioni
Fuzzy merge by plate (Levenshtein ≤ 2)	Bassa	Unifica letture OCR "vicine"
Voto pesato per OCR confidence	Media	Riduce impatto letture rumorose
Stop precoce su consensus alto	Media	Risparmio computazionale
Post-processing italiano (regex AA000AA, correzione I/O/Q/L/T)	Bassa	Recupera letture parziali su targhe italiane

Tutti questi miglioramenti sono compatibili con l'architettura attuale e possono essere aggiunti incrementalmente.

7. Tecnologie utilizzate

Componente	Tecnologia	Riferimento
Vehicle detection	YOLO11s (Ultralytics) Hailo HEF	Hailo Model Zoo, custom training NGS

Plate detection	YOLO11n (Ultralytics) Hailo HEF	Training Federico, F1=0.955
Tracking	ByteTrack	supervision library
OCR	fast-plate-ocr cct-xs-v1-global	Repository HuggingFace, F1>99%
Runtime ONNX	ONNX Runtime (wheel ARM64)	Microsoft
Runtime Hailo	HailoRT + hailo-apps	Hailo
Hardware	Raspberry Pi 5 (16GB) + Hailo-8 AI HAT+ (26 TOPS)	-

8. Conclusioni

La pipeline v2 rappresenta uno stato dell'arte raggiungibile su hardware Raspberry Pi 5 + Hailo-8 per il caso d'uso NGS2026, senza dipendenze da cloud o API esterne.

Punti di forza:

- 100% on-device, nessuna chiamata di rete
- Plate detection con F1 0.955 su targhe miste europee
- OCR fast-plate-ocr al 86% exact match su targhe miste, ~100% sulle targhe italiane testate (sample UniData, 10/10)
- Tracking + majority voting + merge by plate gestisce sia occlusioni che transiti distinti
- Output strutturato (JSON) e visivo (video annotato) per validazione

Punti aperti (da indirizzare con dati di campo reale):

- Tuning fine dei parametri merge (`MERGE_MAX_GAP_SECONDS`) in base al caso d'uso
- Validazione su video italiano con camera a 2-3m
- Eventuale fuzzy matching per ridurre falsi tracker_id

La pipeline è pronta per la prossima fase: **deployment di test su scenario reale** con camera a 2-3 metri di altezza in scenario semaforico/parcheggio, per validare nel dominio target e tarare i parametri operativi.

Appendice — Codice sorgente

Il codice della pipeline è disponibile nel file `ngs_pipeline_v2.py` e nel repository del progetto:

- Repo: `github.com/Bxster/ngs2026`
- Script: `ngs_pipeline_v2.py`
- Dipendenze Python: `hailo_platform` , `opencv-python` , `numpy` , `supervision` , `fast-plate-ocr[onnx]`
- Lancio:

```
source ~/hailo-apps/setup_env.sh
PYTHONIOENCODING=utf-8 LANG=C.UTF-8 python3 ngs_pipeline_v2.py
```
